

Cahier des Charges : Microservice d'Intelligence Artificielle (NIDS)

A. Contexte et Rôle du Microservice

- Objectif : Développer, entraîner et déployer un modèle de Machine Learning / Deep Learning capable de classifier le trafic réseau.
- Rôle dans l'architecture : Agir en tant que microservice d'inférence indépendant. Il reçoit les caractéristiques (features) du trafic réseau depuis le backend Node.js, les analyse, et retourne une prédiction instantanée (Normal vs. Attaque).

B. Spécifications des Données (Data Pipeline)

- Format d'entrée : Vecteurs de données numériques/catégorielles extraits par Zeek ou Suricata (ex: IP source/destination, ports, durée de connexion, nombre d'octets, flags TCP).
- Prétraitement (Preprocessing) : Normalisation/Standardisation des données numériques. Encodage (One-Hot Encoding) des variables catégorielles (ex: protocoles TCP/UDP/ICMP).
- Dataset d'entraînement : Utilisation de jeux de données standards de cybersécurité (ex: NSL-KDD, CICIDS2017, ou UNSW-NB15) pour apprendre au modèle à reconnaître les signatures d'attaques telles que le DoS ou le Port Scanning.

C. Choix Technologiques et Algorithmiques

- Langage & Frameworks : Python, avec Scikit-Learn (pour le Machine Learning classique) ou TensorFlow (pour le Deep Learning).
- Algorithmes recommandés (à tester et comparer) : Supervisé (Random Forest, XGBoost, ou SVM) et Non-supervisé (Isolation Forest ou Autoencodeurs).
- Exposition API : Utilisation de FastAPI ou Flask pour créer une API REST légère (ou gRPC pour plus de performances) afin de communiquer avec le backend Node.js.

D. Besoins Non-Fonctionnels & Métriques de Performance

- Précision (Accuracy) : Taux de détection global supérieur à 90 %.
- Rappel (Recall) & F1-Score : Minimisation absolue des faux négatifs (laisser passer une attaque est plus grave que de bloquer un trafic légitime).
- Taux de Faux Positifs (FPR) : Doit être maintenu le plus bas possible pour éviter de polluer le Dashboard de l'administrateur avec de fausses alertes.
- Latence d'inférence : Le temps de réponse du microservice pour analyser un vecteur de caractéristiques doit être de l'ordre de la milliseconde pour garantir le temps réel.

E. Cycle de Vie du Modèle (MLOps)

- Sauvegarde du modèle : Exportation du modèle entraîné via des formats standard (ex: .pkl avec joblib/pickle pour Scikit-Learn, ou format SavedModel pour TensorFlow).
- Mise à jour : Capacité à ré-entraîner le modèle périodiquement avec de nouvelles données (fichiers .pcap capturés) sans interrompre le service de détection.